Applied Project Task 1: System Blueprint and Project Planning

COS40007, CL03 Monday 14:30, Aman Abidi

Student name	Student Number	Contribution
Alexander Forster	103988837	For this report I contributed by completing the MLOps and DevOps integration section of the report with the help of Minh who created the high level diagram. I also contributed through the creation of the github repository in the project classroom.
BunTree Choup	105954283	For this report I cover the first part group agreement and project plan and check the format
Minh Cao	104179289	For this report, I completed the System Blueprint Diagram part and contributed with Alex on the MLOps high-level diagram
Changalwala Shanmukha Siddharth	104346012	For this report, I completed the Project Management setup, creating user stories, the jira project creation and configuration

Github Group Link
https://github.com/COS40007-2026-Classrooms/CL03_G05

1. Group Agreement & Project Plan

1.1 Team Roles and Responsibilities

Member	Role	Responsibilities
BunTree Choup	Project Manager / Scrum Master	Part 1 — Group Agreement & Sprint Plan
Minh Cao	ML Engineer / Data Engineer	Part 2 — System Blueprint Diagram and Also does — MLOps high-level diagram
Alexander Forster	DevOps Lead	Part 3 — MLOps & DevOps Design
Changalwala Shanmukha Siddharth	Documentation Lead	Part 4 — Project Management Setup

1.2 Communication Strategy

- **Meeting Cadence:** Weekly team meeting every Sunday via Discord(40 minutes)
- **Async Comms:** Daily async updates posted in a shared WhatsApp / Discord group channel
- **Document Sharing:** All shared documents stored in Google Drive (shared folder per sprint)
- **Code Collaboration:** GitHub used for all code collaboration and version control
- **Response Expectations:** Team members expected to respond to messages within 24 hours on weekdays

1.3 Conflict Resolution Approach

- Raise any concerns in the weekly meeting first — open discussion before escalation
- Majority vote used when the team cannot reach consensus after discussion
- If conflict persists, the Scrum Master facilitates a structured retrospective
- As a last resort, the tutor is consulted for mediation

1.4 On-Track Behaviors

- Completing assigned tasks before sprint deadlines
- Actively participating in standups and providing status updates
- Communicating blockers early rather than waiting until the last moment
- Reviewing teammates' pull requests and providing constructive feedback

1.5 Off-Track Behaviors

- Missing meetings without prior notice
- Failing to complete assigned tasks without communicating blockers
- Making major code or design changes without team discussion

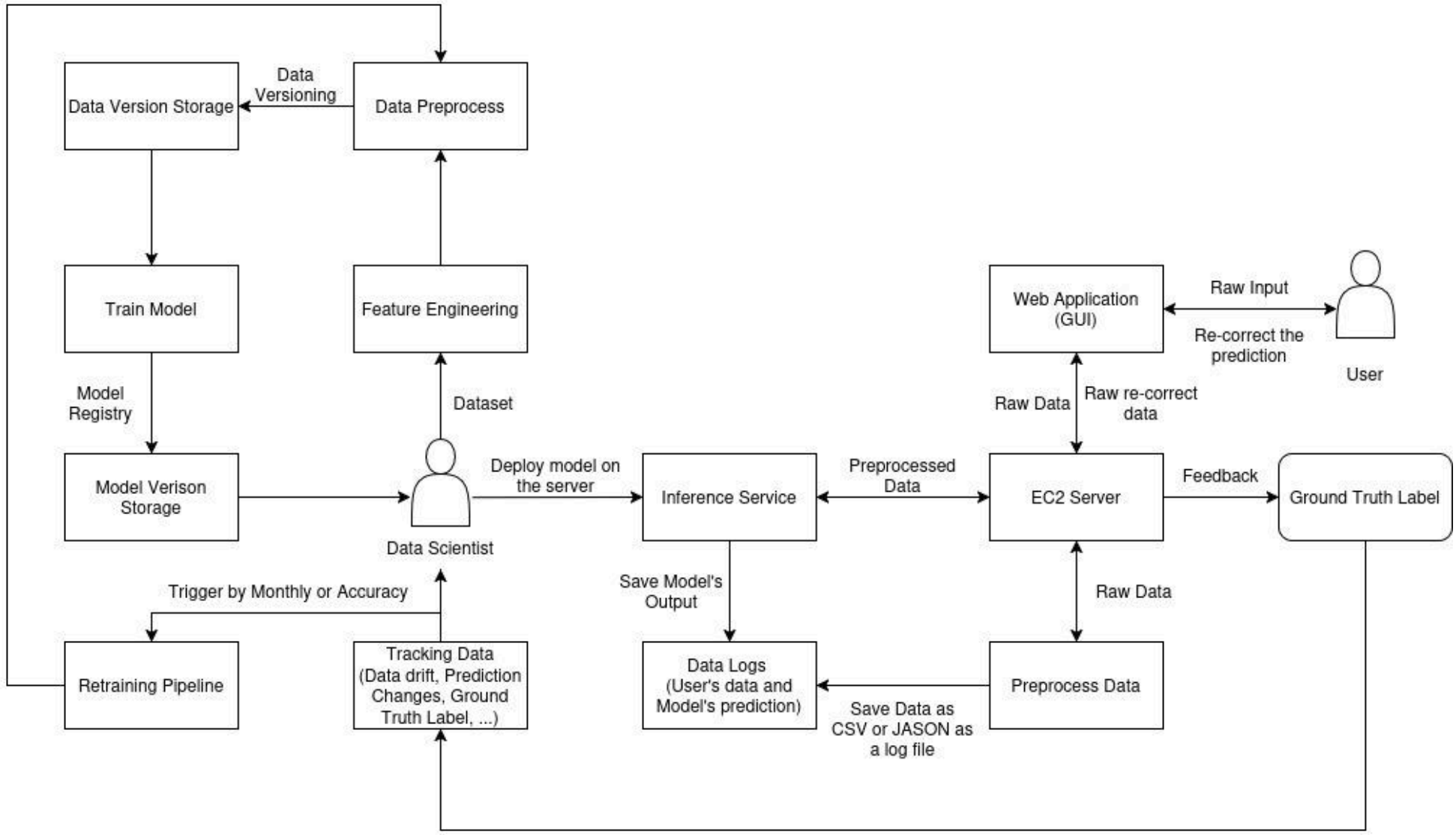
1.6 Team Expectations

#	Expectation
1	Every member contributes meaningfully to each sprint and communicates progress regularly.
2	Code and documentation are reviewed by at least one other team member before merging or submitting.
3	The group submits work as a unified team — all members check the final submission before upload.

1.7 Sprint Plan & Timeline

Sprint	Dates	Goals & Deliverables	Owner(s)
Sprint 1	Week 5–6	Project ideation, dataset selection, EDA, system blueprint diagram, group agreement, GitHub repo setup	All members
Sprint 2	Week 7–8	Data preprocessing pipeline, feature engineering, baseline ML model training, initial evaluation	M2, M3
Sprint 3	Week 9–10	Model optimization, hyperparameter tuning, MLOps integration (monitoring, drift detection concepts)	M2, M4
Sprint 4	Week 11–12	UI / integration layer, CI/CD design, final testing, full documentation, submission preparation	All members

2. System Blueprint Diagram: A diagram showing system components, data flow, and ML model integration with an explanation relevant to your applied project.

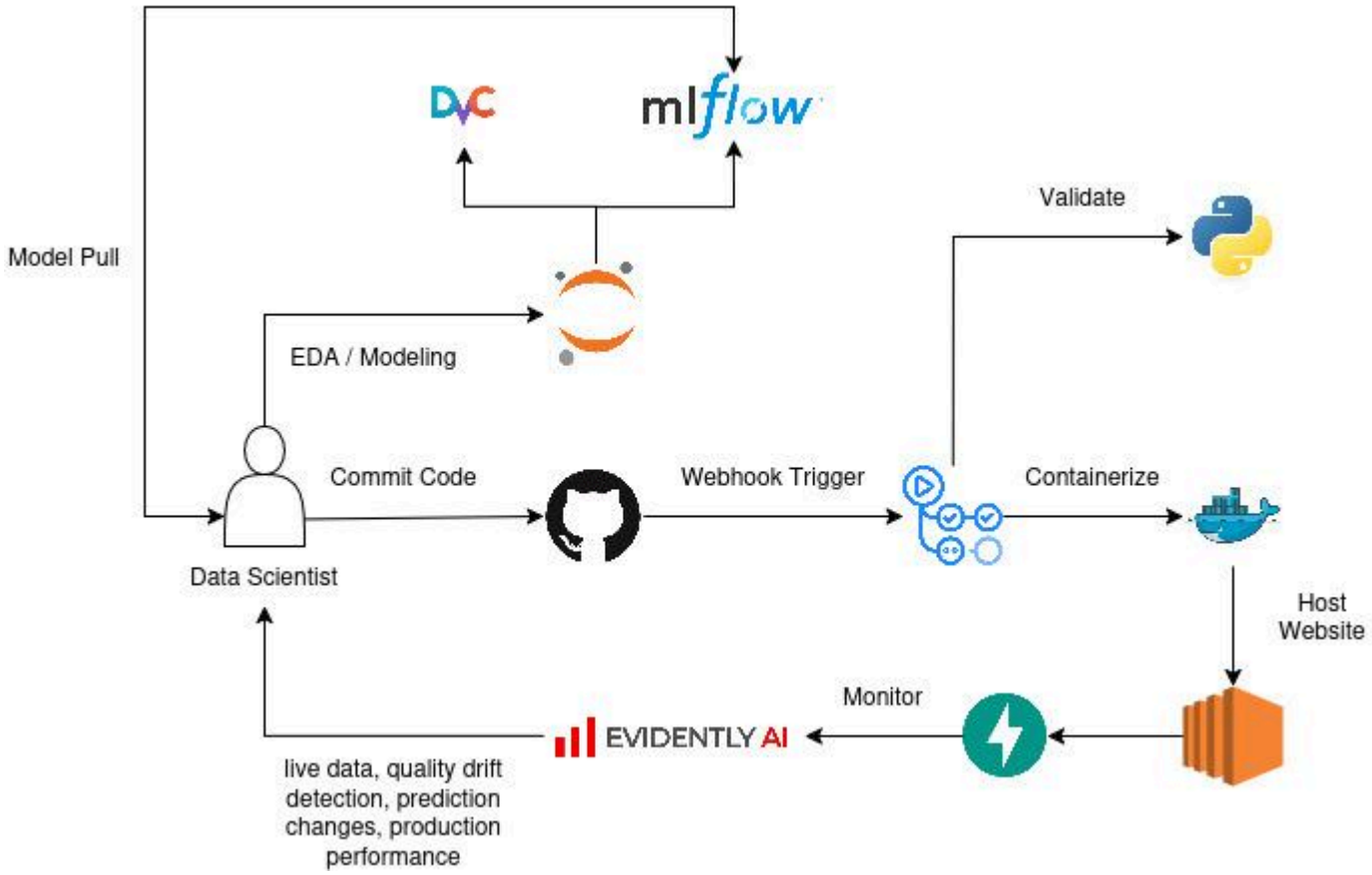


The data flow of the system begins with the collection of data from two primary sources, including historical datasets and real-time user inputs provided through the web application interface. This data first passes through a data ingestion stage, where it is organized and prepared for further processing. It then enters the data preprocessing stage, in which essential transformations are applied, such as handling missing values, correcting inconsistencies, and standardizing formats to ensure data quality and consistency. Following preprocessing, the system performs feature engineering, where meaningful features are derived from the raw input variables to better represent the underlying patterns related to obesity levels. These engineered features are then used to train the machine learning model, allowing it to learn relationships between lifestyle factors and predicted outcomes.

Once trained, the model is stored along with its associated metadata, including performance metrics and configuration details, to enable reproducibility and comparison with future model versions. The trained model is then deployed as part of an inference service, which is integrated into a web application that allows users to input their personal data and receive predictions. During operation, the system continuously collects incoming data, prediction outputs, and any user-provided corrections or feedback. This information is monitored to assess data quality, detect changes in input distributions, and identify potential degradation in model performance over time.

The monitoring component plays a critical role in maintaining system reliability by identifying signs of data drift or shifts in prediction behavior. When such changes are detected, or when a predefined schedule is reached, the system triggers the retraining process. In this stage, newly collected data, including corrected labels where available, is incorporated into the training pipeline to update the model. The updated model is then evaluated and redeployed, replacing the previous version if it demonstrates improved performance. This continuous feedback loop ensures that the system remains adaptive to evolving data patterns and maintains accurate and reliable predictions over time.

3. MLOps & DevOps Integration (High-Level Design): How MLOps (retraining, monitoring) and DevOps (CI/CD, version control) will fit into your project.



The selected dataset will be split into a ratio of 70/15/15 for training, validation and testing respectively. Whereby the training set will be used to train the model, while the validation set will be used for tuning parameters as well as to determine which models to use. The test set will be used for the final performance evaluation, ensuring that the model deployed is assessed on

unseen data, reducing the potential of over-fitting. To further facilitate this the dataset will be shuffled to prevent bias through the ordering of the data while it is stored. As data is collected during production this will be used to facilitate future retraining cycles, allowing the model to adapt to potential changes in data patterns, reducing model drift and maintaining the model's performance over time. This retraining will be triggered using both a time based as well as performance based approach. With the time based ensuring that the model is updated regularly and performance based to detect when the accuracy of the model shows evidence of decline or presence of data drift. As the dataset is somewhat stable the time based retraining can occur on a monthly basis, reducing computing cost while still maintaining a reasonably accurate model.

In order to properly maintain the model's performance a tool called Evidently AI will be used to provide live data, predictive performance metrics as well as drift detection to the data scientist. As more data comes available the statistical properties will be compared to that of the training dataset, if a significant enough discrepancy arises in the feature distributions or categorical frequencies then retraining may be required. While concept drift will be measured by tracking the model's performance over time. Where, if the prediction quality consistently is in decline and the system is stable then it may indicate that the relationship of the input features and the target have changed, indicating that retraining may be required once a threshold has been reached.

To ensure the sustainability of the project, version control will be implemented through the use of GitHub, which will allow for multiple branches to be used for continuous development as well as to roll back to a previous version of the model if required. CI/CD pipelines will be handled through GitHub actions, automatically testing, validation and releasing the changes in a manner that is safe and efficient. The Github action in this case will trigger a python script which will test and validate the model. The model will then be wrapped in a docker container and then deployed on an EC2 server instance. This reduces the amount of manual effort required for the data engineer to deploy the model as well as improves reliability through ensuring that the model is properly validated.

While code versioning is handled by GitHub the model versioning will be handled by MLflow, a version control software specifically for machine learning models. This allows for the data scientist to easily adjust and tune the model and then pull the desired model for deployment. Enabling the roll back to a previous model version if the new model is found to perform poorly. Further as an extension to this the dataset will also have a form of version control through the use of DVC, which will enable the data scientist to be able perform feature engineering and normalisation on the dataset safely due to the version control software. This integrated versioning approach ensures reproducibility, traceability as well as deployment reliability.

4. Project Management Setup (Agile Tools): A Jira (or Trello or others) board with key user stories and a Confluence/Notion/Word page for project documentation.

a. User Stories

The following user stories define the requirements of the Obesity Classification AI System from the perspective of each key stakeholder. Each story follows the standard Agile format: As a [user], I want [goal] so that [benefit].

- **US-01** As a clinician, I want to input a patient's physical measurements and lifestyle data into the system so that I receive an obesity classification to support my clinical diagnosis. Priority: High | Sprint: Sprint 4 | Assignee: ML Engineer
- **US-02** As a data engineer, I want the preprocessing pipeline to automatically encode and scale new patient data so that model-ready feature vectors are produced consistently. Priority: High | Sprint: Sprint 2 | Assignee: Data Engineer
- **US-03** As a ML engineer, I want the system to automatically retrain the model when prediction accuracy drops below 0.93 so that model performance remains reliable over time. Priority: High | Sprint: Sprint 3 | Assignee: ML Engineer
- **US-04** As a healthcare administrator, I want to view per-class prediction confidence scores alongside each classification so that borderline cases can be flagged for clinical review. Priority: Medium | Sprint: Sprint 4 | Assignee: ML Engineer
- **US-05** As a researcher, I want to see feature importance scores displayed alongside each prediction so that I can understand which patient factors most influenced the classification. Priority: Medium | Sprint: Sprint 3 | Assignee: Researcher / Data Engineer.
- **US-06** As a system operator, I want model performance metrics and input feature distributions to be logged automatically after each prediction batch so that I can detect data and concept drift. Priority: Medium | Sprint: Sprint 3 | Assignee: Data Engineer / System Operator

b. Project Documentation

Project Description

This project develops an end-to-end AI classification system that predicts a patient's obesity level from physical measurements and lifestyle survey data. The system is built on the Obesity Estimation Dataset containing 2,087 samples across 7 balanced obesity categories, from Insufficient_Weight through to Obesity_Type_III. The goal is to deliver a clinically useful tool that supports — but does not replace — healthcare professional judgement.

System Overview

The system pipeline consists of five stages:

- Data Ingestion — raw patient data is loaded and deduplicated (2,087 samples, 17 features after removing 24 duplicate rows)
- Preprocessing — categorical features encoded (label, ordinal, one-hot); numerical features scaled (RobustScaler, MinMaxScaler and Standard Scaler)
- Feature Engineering — 5 domain-informed features created from EDA insights (Weight, family_history_with_overweight, FCVC, Gender, BMI)
- Model Training & Evaluation — 6 Decision Tree variants benchmarked; SVM with RBF kernel tuned via GridSearchCV; SelectKBest feature selection (K=10) and PCA (10 components, 95% variance) applied
- Prediction & Output — patient data is passed through the pipeline and classified into one of 7 obesity levels with a confidence score and top contributing features

c. Key Design Decisions

i. Why BMI was engineered rather than used from the dataset

The raw dataset contained Height and Weight separately but not BMI. EDA showed Weight was the single highest correlated feature with obesity class ($r \approx 0.39$). BMI combines both measurements into the WHO clinical obesity metric that the class labels are actually derived from. After engineering, BMI achieved a Gini importance

of 0.79, responsible for 79% of the Decision Tree's predictive power. This was the single most impactful design decision in the project.

ii. Why a tuned SVM was chosen as the primary model

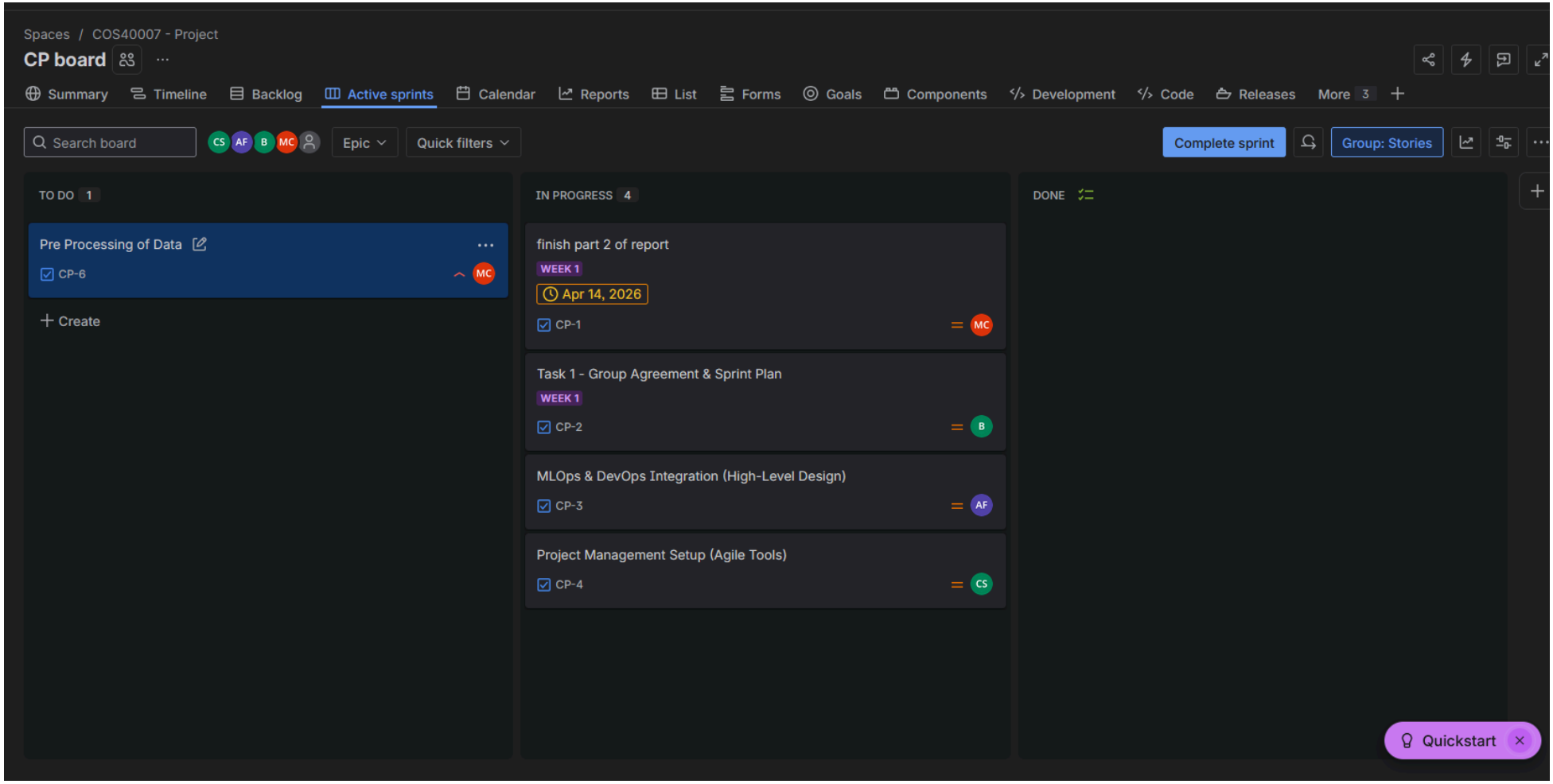
The SVM performs well on this dataset because the problem involves complex, non-linear relationships between lifestyle features, such as eating habits and physical activity, and obesity levels. Using an RBF kernel, the SVM can effectively map these features into a higher-dimensional space where the classes become more separable. The presence of engineered features further enhances class boundaries by capturing meaningful patterns, allowing the model to achieve high discrimination across the seven obesity categories. Additionally, SVM is known for its strong generalization capability, which is reflected in the high and consistent cross-validation accuracy, indicating that the model avoids overfitting while maintaining excellent performance on unseen data.

iii. Why SelectKBest was preferred over PCA

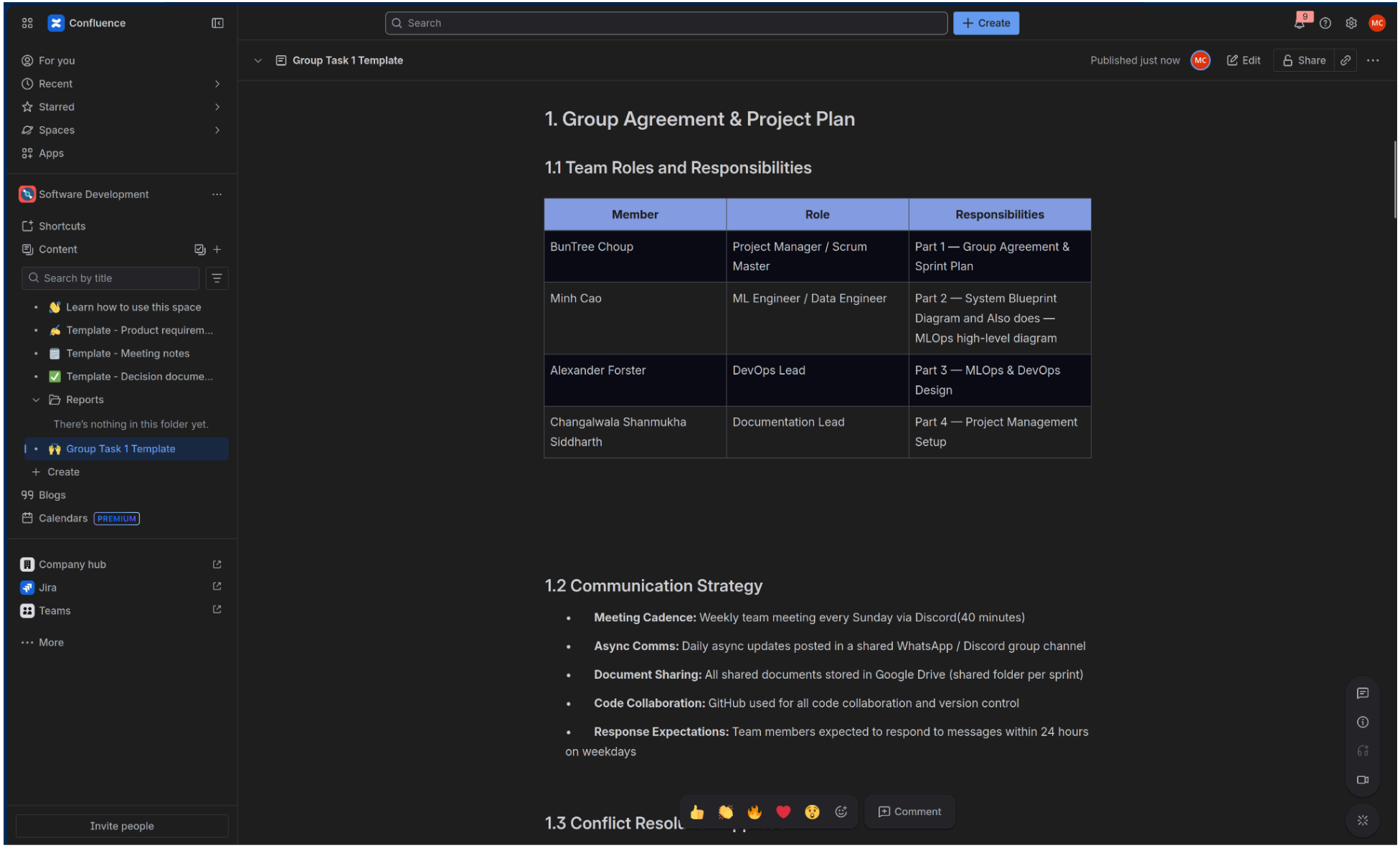
SelectKBest was preferred over PCA because it significantly outperformed PCA in both accuracy and stability while preserving meaningful feature interpretation. With SelectKBest, the model achieved a test accuracy of 0.9713 and strong cross-validation performance (0.9698 ± 0.0161), whereas PCA resulted in a much lower test accuracy of 0.7959 and highly variable cross-validation scores (0.7969 ± 0.1132). This indicates that PCA likely removed or diluted important discriminative information by projecting the data into abstract components. In contrast, SelectKBest directly selects the most relevant original features, such as Weight and BMI, which are strongly linked to obesity levels. This not only improves model performance but also aligns better with SVM, which benefits from clear, informative feature boundaries rather than transformed feature spaces.

iv. Responsible AI considerations

BMI is an imperfect clinical metric that does not account for muscle mass, ethnicity, age, or bone density. Athletes with high muscle mass may be incorrectly classified as overweight. The family_history_with_overweight feature introduces a genetic proxy that requires careful governance in any commercial deployment. All predictions should be treated as decision-support tools requiring clinical review, particularly for borderline cases where the top-2 class confidence scores are within 10% of each other.



Jira Board Screenshot, with assignees for different tasks



Report on the Confluence